

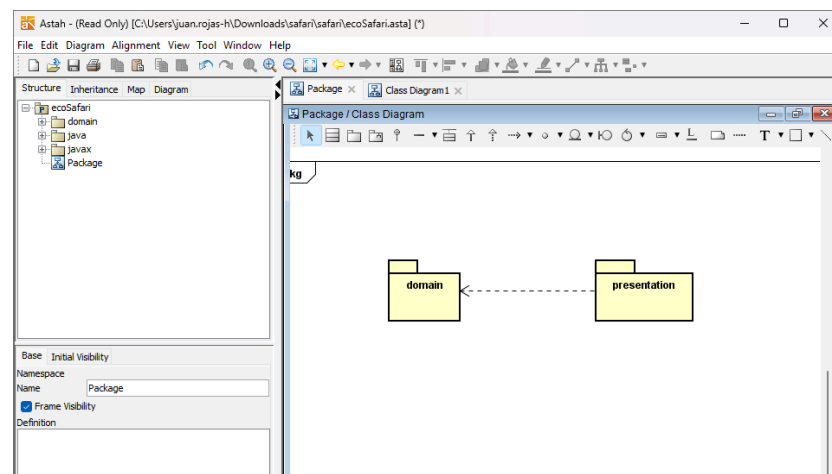
PARTE I. Conociendo el proyecto

1. En el directorio descarguen los archivos contenidos de safari.zip. Revisen el código: (a) ¿Cuántos paquetes tiene? (b) ¿Cuál es el propósito del paquete presentación? (c) ¿Cuál es el propósito del paquete dominio?
 - a. Hay 3 paquetes
 - b. Es para hacer la GUI del proyecto
 - c. Contener las clases e interfaces que representan los conceptos, reglas de negocio y entidades principales.
2. Revisen el paquete de dominio, (a) ¿Cuáles son los diferentes tipos de componentes de este paquete? (b) ¿Qué implica cada uno de estos tipos de componentes?
 - a. Hay clases concretas, clases abstractas e interfaces
 - b. Implican:
 - Clase concreta (EcoSafari, Elephant): Se pueden instanciar directamente (crear objetos con new) y contienen la implementación completa de sus métodos y atributos.
 - Clase abstracta (Organism): Sirve como plantilla o base para otras clases, no se puede instanciar directamente, puede definir atributos y métodos comunes, así como métodos abstractos que las subclases concretas están obligadas a implementar.
 - Interfaz (Entity): Define un contrato o conjunto de métodos/comportamientos que las clases deben implementar. No tiene estado (atributos de instancia) ni implementación de métodos.
3. Revisen el paquete de presentación, (a) ¿Cuántos componentes tiene? (b) ¿Cuántos métodos públicos propios (no heredados) ofrece?
 - a. Tiene solo 1 componente el cual es una clase concreta (EcoSafiriGUI)
 - b. Se tiene 2 métodos publicos
4. Para ejecutar un programa en java, (a) ¿Qué método se debe ejecutar? (b) ¿En qué clase se encuentra?, **Deben ejecutar la aplicación java, no crear un objeto como lo veníamos haciendo**
 - a. Se debe ejecutar el main del paquete presentación
 - b. Se encuentra en la clase EcoSafariGUI la cual esta en el paquete presentation
5. Ejecuten el programa. (a) ¿Qué funcionalidades ofrece? (b) ¿Qué hace actualmente? (c) ¿Por qué?
 - a. Ofrece el Tic-tac
 - b. No hace nada
 - c. Porque no se han implementado

B Realizando ingeniería inversa.

1. Consulten el significado de las palabras package e import de java. (a) ¿Qué es un paquete? (b) ¿Para qué sirve? (c) ¿Para qué se importa? (d) Expliquen su uso en este programa.
 - a. carpeta o directorio que almacena archivos de código fuente agrupados bajo un mismo nombre. Facilita la modularidad, permitiendo separar un proyecto grande en partes más pequeñas y manejables.

- b. Sirve para organizar el código de manera lógica y evitar que todo esté en un solo archivo. Ayuda a prevenir conflictos de nombres entre funciones o clases creadas por diferentes programadores o librerías. Permite reutilizar código fácilmente en otros proyectos o partes de la misma aplicación.
 - c. Se importa para cargar y usar las funciones, clases o variables que están guardadas dentro de ese paquete en otro archivo.
 - d. Estoy creando dos paquetes presentation, domain que son paquetes creados propios del programa además se está diciendo con los imports que se van a usar otros paquetes (paquetes propios de java)
2. Revisen el contenido del directorio de trabajo y sus subdirectorios. (a) Describan su contenido. (b) ¿Qué coincidencia hay entre paquetes y directorios?
 - a. El directorio principal tiene tres directorios donde están los dos paquetes y la documentación y en el directorio de documentación tiene varios sub directorios que están hechos para dividir la documentación
 - b. La coincidencia principal entre paquetes y directorios es que ambos se utilizan para organizar y estructurar elementos de forma jerárquica, funcionando esencialmente como "carpetas" para evitar conflictos de nombres y mantener el orden.
3. Adicionen al diseño la arquitectura general con un diagrama de paquetes en el que se presente los paquetes y las relaciones entre ellos. Consulte la referencia en Moodle. En astah, crear un diagrama de clases (cambiar el nombre por Package Diagram)



C Realizando ingeniería inversa. Arquitectura detallada

1. Para preparar el proyecto para MDD. Realicen el diseño detallado del paquete de dominio. (a) ¿Qué componentes hacían falta?
 - a. Hacían faltan Entify que es una interfaz y Elephant y una clase abstracta
2. Realicen el diseño detallado del paquete de presentación. (a) ¿Por qué hay dos clases y un archivo .java?
 - a. Porque PhotoEcoSafari es una clase interna (inner class) de EcoSafariGUI: está definida dentro de ella (por eso aparece anidada en el árbol), así puede acceder directamente a sus miembros privados y solo tiene sentido como parte visual de

esa GUI. Al estar anidada en el código, ambas clases quedan en el mismo archivo .java.

3. Adicione la clase de pruebas necesaria para BDD en un paquete independiente de test. (No lo adicione al diagrama de clases) (a) ¿Qué clase debe importar? (b) ¿Por qué?
 - a. import dominio.Elephant;
 - b. Elephant vive en el paquete dominio y la prueba vive en test entonces Java no lo reconoce automáticamente

PARTE II. Desarrollando el proyecto

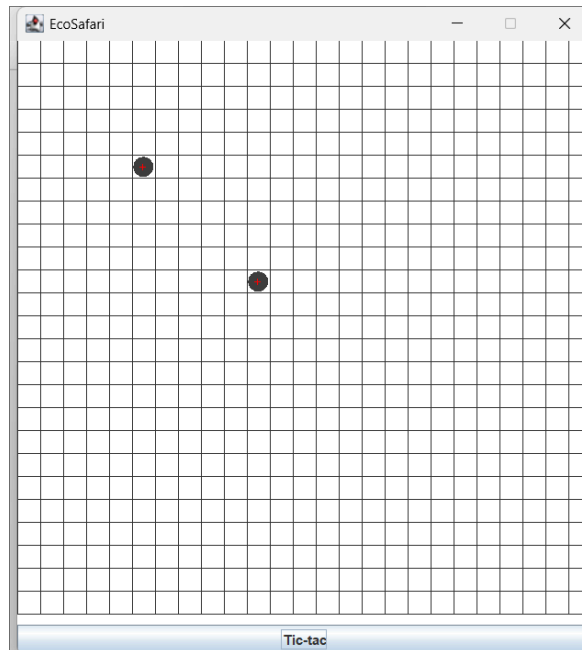
A Ciclo 1. Iniciando con leones

1. Estudie la clase EcoSafari. (a) ¿Qué tipo de colección usa para albergar cosas? (b) ¿Puede tener elefantes? (c) ¿Por qué?
 - a. Usa un arreglo bidimensional (matriz) de Entity
 - b. Si
 - c. Si por el polimorfismo. gracias al mecanismo de herencia de Java, cualquier clase que sea subtipo de Entity puede guardarse en una celda de este arreglo
2. Estudie el código asociado a la clase Elephant , (a) ¿de qué color es? (b) ¿qué forma usa para pintarse? (c) ¿cómo se mueve ? (d) ¿qué componentes definen la clase Elephant? Justifique sus respuestas.
 - a. No tiene un único color fijo: depende de su energía. Si `getEnergy()>=80` = `Color.DARK_GRAY` (gris oscuro) Si `getEnergy()<80` = `Color.LIGHT_GRAY` (gris claro)
 - b. Usa la forma `ROUND` (redonda)
 - c. Se mueve de la siguiente manera
 - Si no ha actuado todavía en este paso (`!hasActed`) y logra moverse (`move(1,1)`) un desplazamiento en diagonal.
 - entonces pierde 10 puntos de energía (`changeEnergy(-10)`) si su energía llega a 0, desaparece (`disappear()`).
 - Al final sin importar si se movió o no marca `hasActed = true` para no volver a actuar en el mismo tic.
 - d. Los componentes son:
 - Atributos propios:
 1. `habitat` (final): referencia al EcoSafari donde vive
 2. `hasActed` (boolean): bandera de control para saber si ya actuó en el ciclo tic-tac actual.
 - Constructor: `Elephant(EcoSafari habitat, int row, int column)`
 - Métodos propios
 1. `getHabitat()`
 2. `getColor()`
 3. `shape()`
 4. `tic()`
 5. `tac()`
 - Herencia
 1. Extends `Organism`. Hereda comportamiento común de organismos vivos probablemente `move()`, `getEnergy()`,

changeEnergy(), disappear(), ya que Elephant los usa pero no los define aquí).

2. implements Entity: cumple el contrato de ser una entidad de la cuadrícula
3. Elephant por ser un Organismo , (a) ¿qué atributos tiene? (b) ¿qué puede hacer (métodos)? (c) ¿qué decide hacer distinto (sobre-escritura)? (d) ¿qué no puede hacer distinto (métodos finales)? (e) ¿qué debe aprender a hacer (métodos abstractos)? Justifique sus respuestas.
 - a. Solo un atributo y es energy que es privado y por esto Elephant no accede directamente a él
 - b. Son:
 - changeEnergy(int value) cambia la energía pero manteniendo la dentro los límites validados [0, 100]
 - changeEnergy(float percentage) Este método calcula un cambio proporcional de energía basado en un porcentaje y luego reutiliza el método changeEnergy(int)
 - getEnergy(): Devuelve la cantidad de energía
 - isOrganism() retorna true
 - c. Nada. Elephant no sobrescribe ningún método de Organism
 - d. Todos los métodos de Organism son final el manejo de energía es una regla de negocio común e invariante para todo organismo (mantener energy entre 0 y 100),
 - e. Ninguno. Organism no declara ningún método abstracto
4. Por comportarse como un Entity , (a) ¿qué debe hacer? (b) ¿qué decide hacer distinto? (c) ¿qué no puede hacer distinto? (d) ¿qué debe aprender a hacer? Justifique sus respuestas.
 - a. Son:
 - public void tic();
 - public abstract Color getColor();
 - public abstract EcoSafari getHabitat();
 - b. Redefine
 - shape(): El default retorna SQUARE, pero Elephant lo sobrescribe para retornar ROUND.
 - tac(): El default no hace nada ({}), pero Elephant lo sobrescribe para resetear hasActed = false.
 - c. Los default de una interfaz técnicamente pueden sobrescribirse sin restricción desde la interfaz). Pero Elephant decide marcar como final sus propias implementaciones
 - d. los métodos abstractos son precisamente los que la clase "debe aprender a hacer" porque no traen implementación heredada
 - tic().
 - getColor()
 - getHabitat()
5. De acuerdo a lo anterior un Elephant , (a) ¿Cómo actúa?

- a. Se pinta como un círculo (`shape() = ROUND`). Su color indica su estado de salud: gris oscuro si tiene 80% de energía o más, gris claro si tiene menos. Un Elephant nace con 100% de energía (heredado de `Organism`) y actúa siguiendo el ciclo tic-tac de la simulación:
 - i. Durante `tic()`:
 1. Si aún no ha actuado en este paso (`hasActed == false`):
 - a. Intenta moverse en diagonal una celda hacia el noreste. Este movimiento solo tiene éxito si la posición destino está dentro de la cuadrícula (`isInside`)
 - b. Si el movimiento fue exitoso, pierde 10 puntos de energía `changeEnergy(-10)`, con el límite inferior de 0 garantizado por `Organism`.
 - c. Si tras perder energía llega a 0, desaparece de la pradera
 2. Independientemente de si se movió o no, marca `hasActed = true` para no volver a actuar
 - ii. Durante `tac()`:
 1. Simplemente resetea `hasActed = false`, dejándolo listo para actuar de nuevo en el siguiente ciclo tic-tac.
6. Ahora vamos a crear dos elefantes en diferentes posiciones (5,5) (10,10) , llámelos dumbó y babar, usando el método `someEntities`. Ejecuten el programa, (a) ¿Qué pasa con los elefantes? (b)¿Por qué? (c) Capturen una pantalla significativa.
 - a. Aparecen los elefantes en el tablero. El elefante dumbó en la posición (5,5) y a babar en (10,10) pero se van a quedar quietos
 - b. Porque todavía no está implementado `ticTac()`
 - c.



7. Diseñen, construyan y prueben el método llamado ticTac() de la clase EcoSafari .

```
/**
 * Advances the simulation by one time step
 */
//First, all entities execute their tic() action
//Then, all entities execute their tac() actions
public void ticTac(){
    for (int i=0; i<SIZE; i++){
        for (int j=0; j<SIZE; j++){
            Entity e = get(i, j);
            if (null !=e){
                e.tic();
            }
        }
    }
    for (int i=0; i<SIZE; i++){
        for (int j=0; j<SIZE; j++){
            Entity e = get(i, j);
            if (null !=e){
                e.tac();
            }
        }
    }
}
```

```
public class EcoSafariTest
{
    private EcoSafari safari;

    @BeforeEach
    public void setUp()
    {
        safari = new EcoSafari();
    }

    @Test
    public void testTicTacMuevePosicionDelElephant()
    {
        Elephant dumbbo = new Elephant(safari, 5, 5);

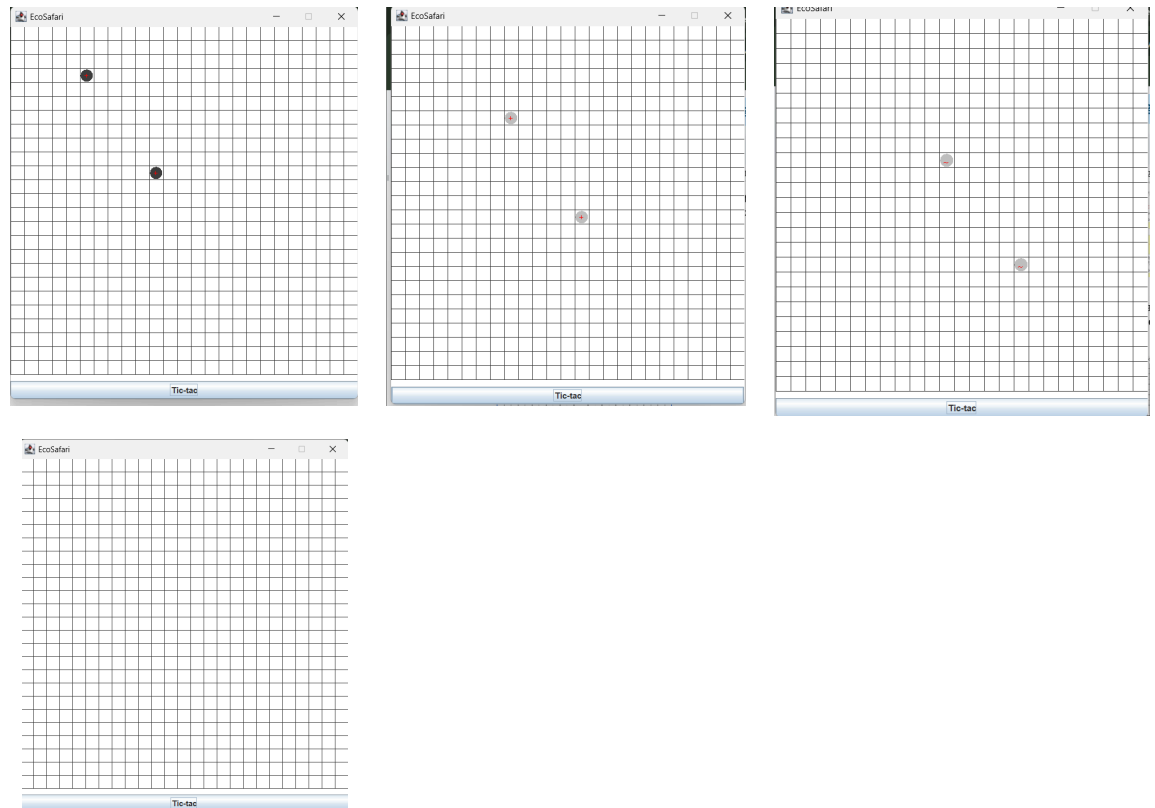
        safari.ticTac();

        int[] posicion = safari.find(dumbbo);
        assertEquals(6, posicion[0]); // fila
        assertEquals(6, posicion[1]); // columna
    }

    @Test
    public void shouldLessEnergy()
    {
        Elephant dumbbo = new Elephant(safari, 5, 5);
        safari.ticTac();
        assertEquals(90, dumbbo.getEnergy());
    }
}
```

8. (a) ¿Cómo quedarían dumbbo y babar después de 3, 5, 10 tic-tac? Ejecuten el programa. Capturen pantallas significativas en los momentos correspondientes. (b)¿Es correcto?

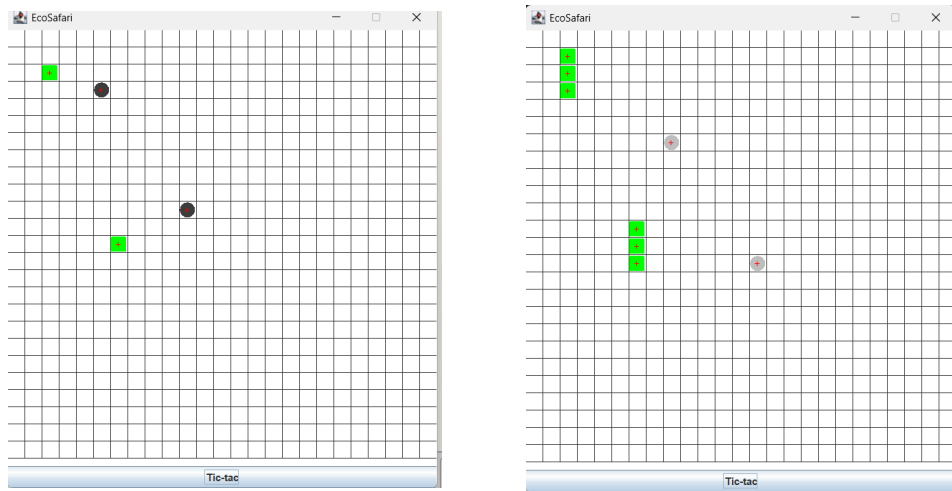
a. El elefante va a desaparecer



b. Porque su energia llega a 0 ya que $10 \times 10 = 100$ y $100 - 100 = 0$

B Ciclo 2. Incluyendo arbustos

1. Para implementar esta nueva clase Organism (a) ¿cuáles métodos se sobre-escriben (overriding)?
 - a. Los métodos que se deben sobrescribir son los de la interfaz Entity getColor(), getHabitat(), tic()
2. Diseñen, construyan y prueben esta nueva clase. (Mínimo dos pruebas de unidad)
3. Adicione una pareja de arbustos , de nombre mopane y acacia , (a) ¿Cómo quedarían después de 3 Tic-tac? Ejecuten el programa y hagan 3 clics en el botón. Capturen una pantalla significativa. (b) ¿Es correcto?
 - a. Quedarían 6 arbustos ya que en el 2 tic cada uno de los arbustos tiene un hijo en el norte y en el tercero tienen otro hijo en el sur

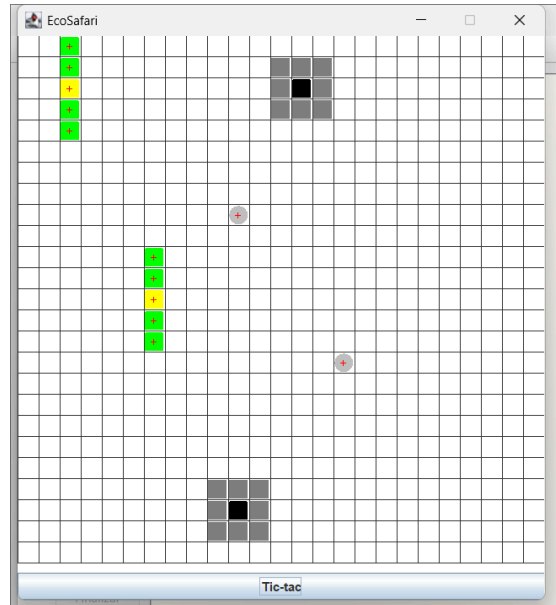


- b. Si es correcto y sigue las reglas de negocio

C Ciclo 3. Adicionando tormentas

1. Para poder adicionar tormentas , (a) ¿debe cambiar en el código de EcoSafari en algo? (b) ¿porqué? (c) ¿debe extender Entity ? (d) ¿por qué?
 - a. Sí, ya que se deben agregar dos métodos y un nuevo atributo que sea un arreglo de booleanos bidimensional
 - b. Para saber si está afectada la casilla y poder pintarla porque actualmente en cada casilla sólo permite una entidad
 - c. Sí, Storm debe implementar la interfaz Entity.
 - d. Porque cumple naturalmente con el contrato de Entity:
 - tic(): la tormenta se desplaza circularmente en diagonal en cada paso este es su comportamiento de actuación por tic.
 - getColor(): el enunciado especifica que su centro es negro y las casillas del lado son grises.
 - getHabitat(): necesita conocer el EcoSafari en el que vive para consultar y modificar celdas.
2. Diseñen , construyan y prueben esta nueva clase. (Mínimo dos pruebas de unidad)

3. Adicionen dos objetos tormentas en EcoSafari , llámenlos thor y tempest , (a) ¿Cómo quedarían después de cuatro Tic-tac? Ejecuten el programa y hagan 5 clics en el botón. Capturen una pantalla significativa. (b) ¿Es correcto?
 - a. Se van moviendo por la pradera incluso una traspasa el mapa asi como lo dice las reglas de negocio

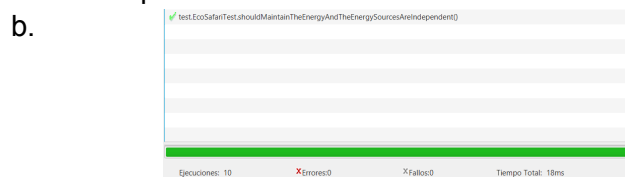


- b. Si es correcto y cumple con las reglas

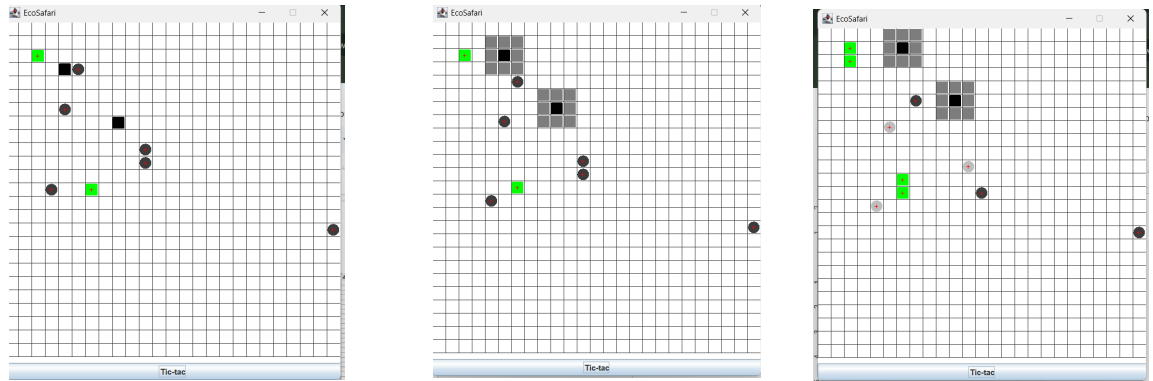
D Ciclo 4. Proponiendo y diseñando : Nuevo elefante

1. Propongan, describan e implementen el nuevo tipo de Elephant . (Mínimo dos pruebas de unidad)

El SickElephant es una subclase de Elephant que representa un elefante enfermo, con dos diferencias frente al normal: se mueve más lento y se desgasta más rápido. Alterna entre moverse y descansar: en un tic-tac avanza una casilla en diagonal, como cualquier elefante, y en el siguiente se queda quieto. Además pierde 20 de energía en cada acción si se mueve y si descansa pierde 10. Cuando su energía llega a 0 o menos, desaparece de la pradera.
2. Considerando una pareja de ellos con el apellido de ustedes. (a) Piensen en otra prueba significativa y expliquen la intención. (b) Codifiquen la prueba de unidad correspondiente y capture la pantalla de resultados de ejecución de la prueba. (c) Ejecute el programa con esa prueba como prueba de aceptación y capturen las pantallas correspondientes.
 - a. comprobar que dos SickElephant no comparten estado y que cada uno sigue su propio ritmo. rayo está en el borde, donde move devuelve false, así que no se mueve, no pierde energía y no avanza su contador. Mientras tanto, casco sí se mueve en el primer tic-tac y descansa en el segundo, con el desgaste correspondiente.



C.



E Ciclo 5. Proponiendo y diseñando : Nueva entidad

El objetivo de este punto es permitir recibir una nueva clase de Entity (no Elephant) en el EcoSafari

1. Propongan, describan e implementen la nueva clase de Entity . (Mínimo dos pruebas de unidad)

Propuesta:

Un salitral, es un depósito mineral que los animales de la sabana usan como fuente de suplemento mineral

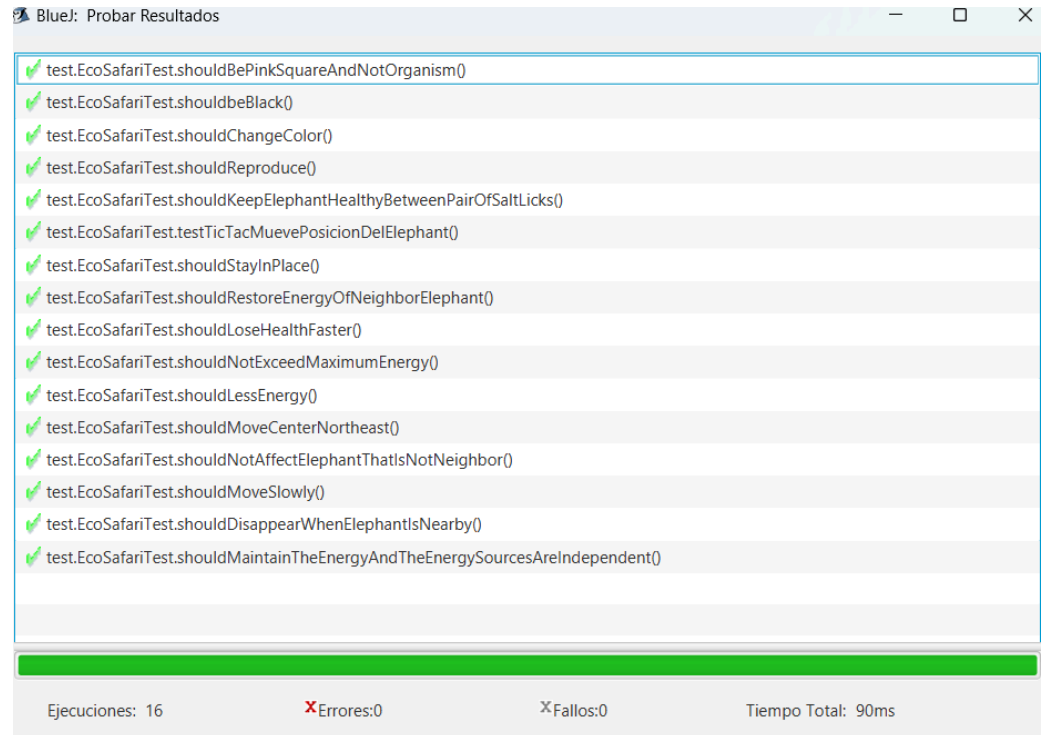
Descripción:

- Es una Entity que no es Organism: no tiene energía, no se mueve y no muere por sí sola.
- En cada tic devuelve +10 de energía a todo organismo en sus 8 celdas vecinas.
- Es cuadrado y rosado (Color.PINK), distinto de los colores ya usados.
- La energía nunca pasa de 100, porque Organism.changeEnergy ya la limita.
- Una tormenta puede destruirlo si su centro pasa por encima, igual que con cualquier otra cosa.

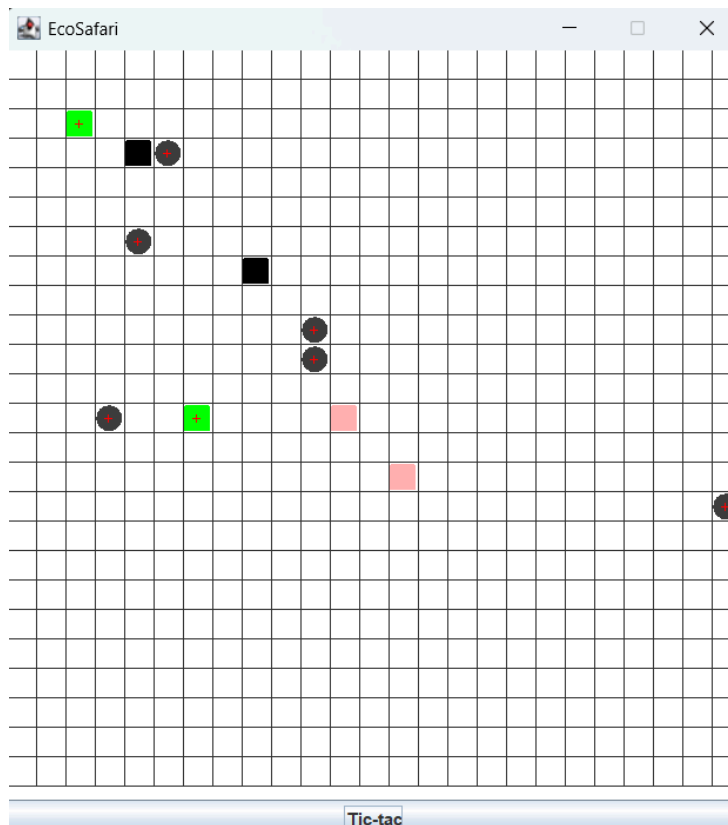
Pruebas:

1. shouldRestoreEnergyOfNeighborElephant: un elefante vecino con 50 sube a 60.
 2. shouldNotAffectElephantThatIsNotNeighbor: a distancia 2 no cambia nada.
 3. shouldNotExceedMaximumEnergy: con 95 queda en 100, no en 105.
 4. shouldStayInPlace: tras 3 ticTac sigue en su celda.
 5. shouldBePinkSquareAndNotOrganism: color, forma y isOrganism().
2. Considerando un par de ellos con el nombre de ustedes. (a) Piensen en otra prueba significativa y expliquen la intención. (b) Codifiquen la prueba de unidad correspondiente y capturen la pantalla de resultados de ejecución de la prueba. (c) Ejecuten el programa con esa prueba como prueba de aceptación y capturen las pantallas correspondientes.

- a. Un par de salitrales llamados cesar y Juan se colocan a lo largo del camino diagonal de un elefante. La intención es comprobar que la ayuda mineral compensa el desgaste del movimiento: el elefante llega a 100 de energía tras 4 tic-tac, mientras uno idéntico sin salitrales baja a 60.
- b. es shouldKeepElephantHealthyBetweenPairOfSaltLicks

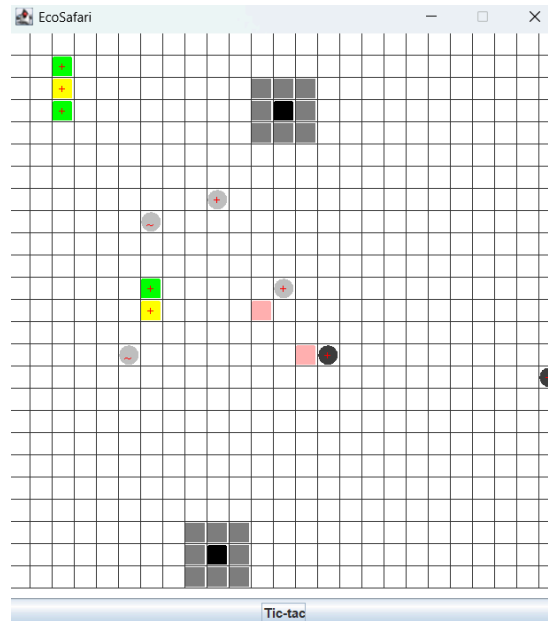


- c. Estado en Tic-tac 0



Lo que se ve el estado inicial (tic-tac 0) con los dos SaltLick rosados ya en su lugar, en (12,11) y (14,13) y nos fijaremos en el elefante en (10,10) y el elefante en (3,5)

Estado en Tic-tac 4

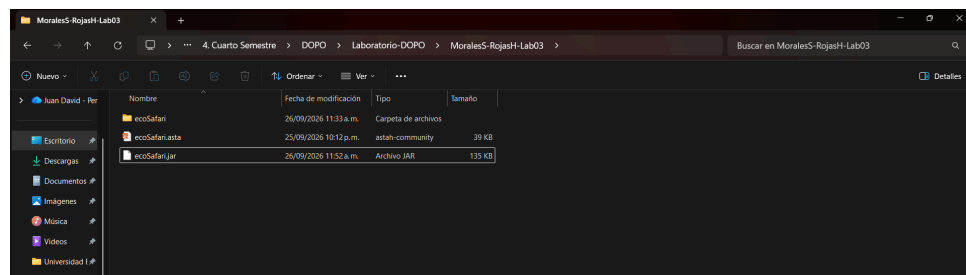


Después de presionar Tic-tac 4 veces podemos ver que el elefante que estaba en la fila 10 y columna 10, ahora está en (14,14) y todavía gris oscuro, mientras que el elefante que estaba en (3,5) ya está en (7,9) y gris claro. Ese contraste es la evidencia de que los SaltLicks le devolvieron energía al elefante que empezó en (10,10).

PARTE IV. Empaquetando el proyecto

1. Revise las opciones de BlueJ para empaquetar su programa en un archivo .jar. Genere el archivo correspondiente. (a) Incluya una pantalla con evidencia de la existencia del archivo.

a.



2. Consulte el comando java para ejecutar un archivo jar. ejecútenlo (a) ¿qué pasa?
 - a. Al ejecutar `java -jar ecoSafari.jar` desde la terminal el programa se ejecuta directamente sin necesidad de abrir BlueJ, mostrando la ventana gráfica del

EcoSafari con el tablero de simulación y el botón "Tic-tac" para avanzar el tiempo.

3. (a) ¿Qué ventajas tiene esta forma de entregar los proyectos? Explique claramente.
 - a. Las ventajas son:
 - El proyecto completo queda empaquetado en un solo archivo, fácil de compartir.
 - Quien reciba el .jar no necesita tener BlueJ instalado solo necesita tener Java instalado.
 - Se ejecuta con un solo comando (java -jar), sin compilar nada manualmente.
 - Mantiene organizados todos los .class y recursos del proyecto en un único archivo, evitando pérdida o desorden de archivos sueltos.
 - Facilita la distribución del software como producto terminado

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)
Cada integrante del equipo invirtió aproximadamente 12 horas en el desarrollo del laboratorio.
2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?
El laboratorio se encuentra finalizado. Se completaron todas las actividades solicitadas en la guía
3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?
La práctica más útil fue la programación en pareja, ya que permitió detectar errores de lógica en tiempo real y compartir conocimiento sobre el manejo de herencia e interfaces, lo cual aceleró la resolución de problemas frente a trabajar cada uno por separado.
4. ¿Cuál consideran fue el mayor logro? ¿Por qué?
El mayor logro fue comprender, a nivel de código, el funcionamiento de las interfaces y la herencia en Java.
5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
El mayor problema técnico se presentó en la parte de la interfaz gráfica (GUI) relacionada con la tormenta, específicamente la zona que quedaba sombreada, ya que no comprendíamos bien su lógica de implementación. Para resolverlo, consultamos documentación y usamos inteligencia artificial (ChatGPT y Claude).
6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
Como equipo, dividimos el trabajo de manera equilibrada para aprovechar mejor el tiempo disponible.
7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.
Se utilizaron las siguientes herramientas como apoyo durante el desarrollo:
 - OpenAI. (2026). *ChatGPT* (versión GPT-4) [Modelo de lenguaje de gran tamaño]. <https://chatgpt.com/>
 - Anthropic. (2026). *Claude* (versión Sonnet) [Modelo de lenguaje de gran tamaño]. <https://claude.ai/new>